

GRANDE ECOLE

LA MANU

CRÉATION ■ MANAGEMENT ■ NUMÉRIQUE

Plateforme LogiChain



1. Contexte et enjeux

L'organisation d'événements de grande envergure (festivals, salons professionnels, rassemblements éco-responsables) implique une gestion logistique complexe, souvent soumise à des conditions de terrain dégradées. La traçabilité du matériel, la gestion des flux de prestataires et l'évaluation de l'impact environnemental en temps réel représentent des défis majeurs.

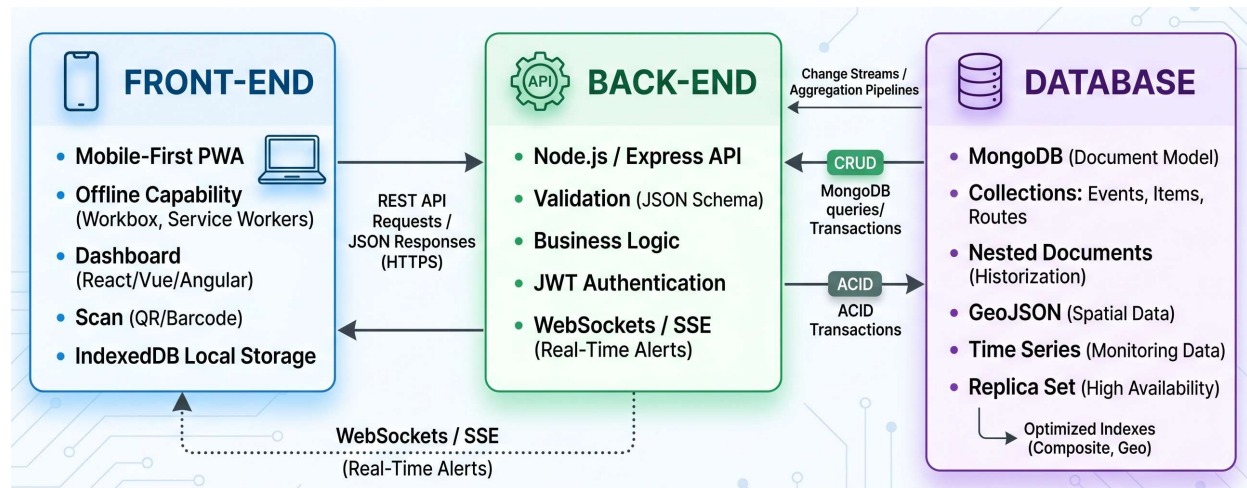
LogiChain est une solution applicative industrielle conçue pour centraliser, tracer et optimiser l'ensemble de la chaîne logistique événementielle, tout en garantissant un fonctionnement fluide aux agents de terrain, même en l'absence de réseau internet.

2. Objectifs du projet

- **Centraliser et sécuriser** la gestion des ressources matérielles et humaines sur une infrastructure de données scalable.
- **Garantir la continuité de service** sur le terrain via une approche *Offline-First*.
- **Fournir des outils d'aide à la décision** et de pilotage basés sur des métriques de performance et d'impact environnemental.
- **Assurer la résilience et la charge** du système face à des pics d'utilisation simultanés (scans massifs lors des phases de montage/démontage).

3. Périmètre technique global

L'application repose sur une architecture moderne découpée en trois grands piliers interconnectés :



Infrastructure et base de données

- **Technologie principale** : MongoDB.
- **Modélisation** : Approche documentaire dénormalisée, utilisation de documents imbriqués (*nested documents*) pour l'historisation, collections de séries temporelles (*Time Series*) pour les données de monitoring, et structures GeoJSON pour la cartographie.
- **Rigueur** : Validation native des schémas, gestion de transactions ACID et optimisation avancée des index (composés, partiels, géospatiaux).

4. Fonctionnalités clés par profil utilisateur

Administrateurs & Responsables logistiques

- Configuration globale de l'événement et découpage cartographique des zones (coordonnées géospatiales).
- Suivi des indicateurs clés (KPI) via un tableau de bord d'agrégation : état des stocks, calcul de l'empreinte carbone consolidée en temps réel, détection des goulots d'étranglement.
- Supervision des transferts de responsabilité et validation des feuilles de route des transporteurs.

Agents de terrain & Prestataires (Interface PWA Mobile)

- Consultation des tâches logistiques et des plannings de livraison assignés.
- Scan des équipements pour validation des étapes de livraison, de mouvement ou de maintenance (mode connecté ou déconnecté).
- Déclaration d'anomalies géolocalisées avec mise à jour immédiate de l'état de l'item.
- Réception de notifications critiques en cas de modification d'urgence sur leur secteur.

5. Contraintes et exigences non-fonctionnelles

- **Programmation Orientée Objet (POO) obligatoire** : L'ensemble du code de l'API doit être structuré autour du paradigme objet. L'usage de fonctions isolées ou de scripts procéduraux est interdit pour la logique métier et l'accès aux données. Les concepts d'encapsulation, d'héritage (si justifié) et de polymorphisme devront être visibles.
- **Découpage architectural strict (N-Tier)** : Pour garantir la maintenabilité et la testabilité du code, les responsabilités de l'API doivent être cloisonnées selon le pattern suivant :
 - **Entity / Model** : Définition des objets métiers de l'application et de leurs règles de validation (en adéquation avec les schémas MongoDB).
 - **Repository** : Couche exclusive d'accès à la base de données. Aucun composant en dehors des repositories n'est autorisé à interagir avec le driver MongoDB ou l'ORM/ODM (ex: Mongoose). C'est ici que sont isolées les requêtes et les pipelines d'agrégation.
 - **Service** : Couche contenant l'intégralité de la logique métier, des calculs (ex: empreinte carbone) et des règles de gestion. Les services orchestrent les repositories et sont totalement indépendants du protocole HTTP.
 - **Controller / Routing** : Point d'entrée de l'application chargé de réceptionner les requêtes, d'appeler les services appropriés et de retourner la réponse HTTP.

- **Architecture REST stricte** : L'API doit adhérer rigoureusement aux contraintes du modèle de maturité de Richardson (niveau 2 minimum).
 - Utilisation correcte et sémantique des verbes HTTP (GET, POST, PUT, PATCH, DELETE).
 - Structure d'URLs basée sur les ressources (ex: /api/v1/events/{id}/items).
 - Utilisation standardisée des codes de statut HTTP (200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 422 Unprocessable Entity, etc.) pour refléter fidèlement le résultat de l'opération.
- **Sécurité par conception (Security by Design)** : Chiffrement des communications, contrôle d'accès strict aux ressources de l'API, et validation systématique de l'intégrité des données à l'entrée de la base de données.
- **Robustesse du mode déconnecté** : Aucune perte de données ne sera tolérée lors des phases de transition réseau. Le mécanisme de synchronisation doit intégrer un système de verrouillage optimiste pour gérer les accès concurrents sur un même document.
- **Performance** : Temps de réponse de l'API inférieur à un seuil critique sur les requêtes fréquentes, garanti par une stratégie d'indexation chirurgicale et validé par des tests de charge intensifs.